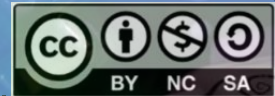


Instrumental Testing en Android





Testing Android Apps

1. **Unit Testing**: Validar la lògica dels mètodes del ViewModel.
2. **Instrumental UI Testing**: Testejar la interacció dels composables de la interfície d'usuari.
3. **Integration Testing**: Validar la integració de components o subsistemes.
4. **End-to-End Testing**: Testar fluxos complets d'interacció de l'usuari amb l'aplicació.
5. **UI Automator**: Proves que impliquen múltiples aplicacions o la interacció amb el sistema operatiu.
6. **Performance Testing**: Mesurar la resposta i eficiència de l'aplicació.
7. **Accessibility Testing**: Validar la disponibilitat per a usuaris amb necessitats especials.
8. **Smoke Testing**: Comprovar que les funcionalitats bàsiques funcionen.



Instrumental UI Testing

- **Propòsit:** Provar la UI de l'aplicació, interactuant amb els components visuals (botons, text fields, etc.).
- **Eines comunes:** Espresso, Jetpack Compose Testing, UI Automator.
- **Exemples:** Comprovar que els botons funcionen correctament, validar que el text es mostra correctament quan s'introdueix a un camp.
- **Avantatges:** Testejant l'experiència de l'usuari real.



Android Espresso framework

- Android Espresso és un framework de proves de codi obert desenvolupat per Google per automatitzar les proves d'interfície d'usuari (UI) en aplicacions Android.
- Està dissenyat per ser ràpid i fiable, gràcies a la seva capacitat de sincronització automàtica amb el fil principal de la interfície d'usuari de l'aplicació.



Android Espresso framework

Components Principals de l'API:

- L'API d'Espresso és petita i fàcil d'aprendre, i es basa en tres accions fonamentals que simulen el comportament d'un usuari real:

- **ViewMatchers** (Trobar alguna cosa): Permeten localitzar una vista dins de la jerarquia de l'aplicació.

Exemple: `withId(R.id.my_button)` o `withText("Acceptar")`

- **ViewActions** (Fer alguna cosa): Permeten interactuar amb la vista trobada.

Exemple: `click()`, `typeText("Hola")` o `scrollTo()`

- **ViewAssertions** (Comprovar alguna cosa): Permeten verificar que el seu estat és el correcte.

Exemple: `matches(isDisplayed())` o `matches(withText("Resultat"))`



Dependències

- És necessari incloure les següents dependències al **build.gradle.kts** (module:app):

```
androidTestImplementation("androidx.test.espresso:espresso-core")
androidTestImplementation("androidx.test.ext:junit" )
androidTestImplementation("androidx.compose.ui:ui-test-junit4" )
androidTestImplementation(libs.androidx.espresso.core)
androidTestImplementation(libs.androidx.junit)
androidTestImplementation(libs.androidx.ui.test.junit4)
```



Dependències

- També es recomana incloure les següents:

```
debugImplementation("androidx.compose.ui:ui-tooling" )  
debugImplementation("androidx.compose.ui:ui-test-manifest" )  
debugImplementation(libs.androidx.ui.tooling)  
debugImplementation(libs.androidx.ui.test.manifest)
```

- Ens permetran fer test de UI amb contexts ficticis evitant errors.



View

- Variables per subscripció dins de la View:

```
@Composable
fun MyView(myViewModel: HelloViewModel, modifier: Modifier = Modifier) {
    val textValue by myViewModel.textValue.observeAsState("")
    val checkBoxMulti by myViewModel.checkBoxMulti.observeAsState(false)
    val counterValue by myViewModel.counterValue.observeAsState(0)
    ...
}
```




View

- A través del **Modifier** definirem un **testTag** pels Composables que volguem testejar.
- Usarem els **testTag** des de la classe de Testing de UI per a referir-nos als elements de la vista.
- És l'equivalent a definir un #id en HTML i usar-lo després al CSS o JS.

```
item{
    Text(
        text = "Hello $textValue!",
        modifier = Modifier
            .testTag("initialText_id")
    )
}

item{
    Text(
        text = "$counterValue",
        modifier = Modifier
            .testTag("counterText_id"),
        color = Color.Green
    )
}
```



View

```
item{  
    Checkbox(  
        checked = checkBoxMulti,  
        // Invertim el valor de la variable al interactuar amb el Checkbox  
        onCheckedChangeListener = {myViewModel.toggleCheckBoxMulti()},  
        modifier = Modifier  
        // Fem que el component Switch sigui responsive ocupi el 40% de l'amplada del component superior  
        .fillMaxWidth(0.20f)  
        .testTag("checkBoxMulti_id"),  
        enabled = true,  
        colors = CheckboxDefaults.colors(  
            uncheckedColor = Color.LightGray,  
            checkmarkColor = Color.Black)  
    )  
}
```



View

```
item{  
    OutlinedTextField(  
        value = textValue,  
        onChange = { myViewModel.setTextValue(it) },  
        label = { Text("Write here...") },  
        modifier = Modifier  
            .testTag("outlinedTextField_id")  
    )  
}
```



View

```
item{
    Button(onClick = { myViewModel.clickIncrement() },
        modifier = Modifier
            .testTag("incrementButton_id")
    ) {
        Text("Counter")
    }
}
```

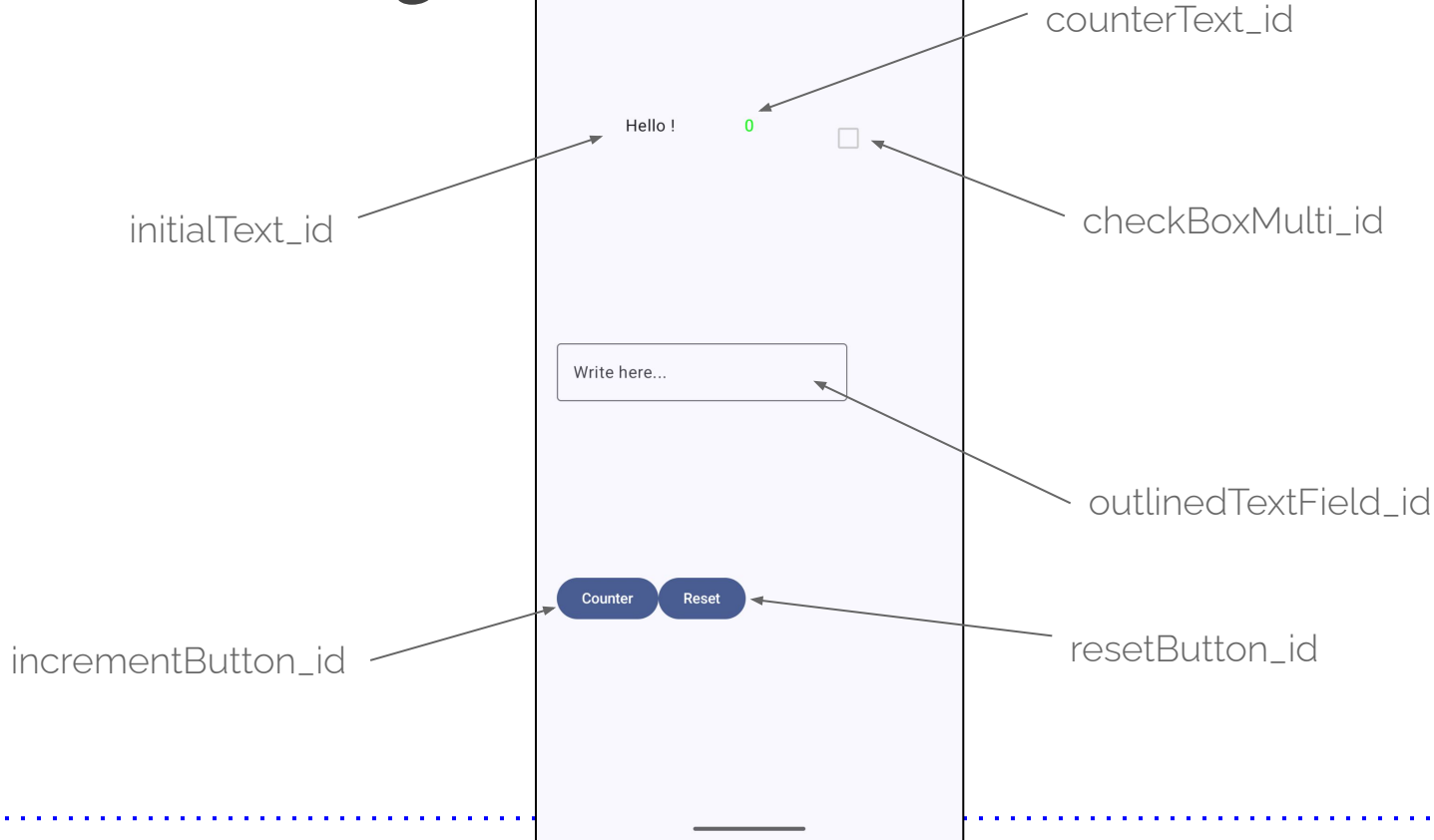


View

```
item{  
    Button (onClick = { myViewModel.resetValues() },  
            modifier = Modifier  
                .testTag("resetButton_id")  
    ) {  
        Text ("Reset")  
    }  
}
```



View *testTag()*





Instrumental UI Testing

- Setting del context dels tests de UI: li especifiquem quina vista volem provar. En aquest cas: *'MainActivity'*.

```
@LargeTest
@RunWith (AndroidJUnit4 :: class)
class ViewInstrumentedUITest {
    @get:Rule
    val composeTestRule = createAndroidComposeRule<MainActivity>()
    ...
}
```

- `@get:Rule` és una anotació especial de JUnit que s'utilitza per aplicar una regla abans de cada test.
- Declarem i definim ***composeTestRule*** per a usar-lo posteriorment a la resta de tests de UI.



Instrumental UI Testing

- Un cop tenim el ***composeTestRule*** definit, podem aplicar accions de **testing sobre els nodes** (elements) de la view.
- Hi ha **dues tipologies de test** que podem usar:
 - **Assertions** (verificacions d'estat)
 - **Actions** (interaccions simulades)



Instrumental UI Testing: assertions

- **Assertions:** aquests **mètodes** comproven que un **node** té un **determinat estat** o propietat.
 - `assertExists()` – Verifica que el node existeix.
 - `assertDoesNotExist()` – Verifica que el node **no** existeix.
 - `assertIsDisplayed()` – Verifica que el node està visible a la pantalla.
 - `assertIsNotDisplayed()` – Verifica que el node **no** està visible.
 - `assertIsEnabled()` – Verifica que el node està activat.
 - `assertIsNotEnabled()` – Verifica que el node està desactivat.
 - `assertIsSelected()` – Verifica que el node està seleccionat.
 - `assertIsNotSelected()` – Verifica que el node **no** està seleccionat.



Instrumental UI Testing: assertions

- ...
- `assertIsFocused()` – Verifica que el node té el focus.
- `assertIsNotFocused()` – Verifica que **no** té el focus.
- `assertIsOn()` – Verifica que un `Switch` o `Checkbox` està activat.
- `assertIsOff()` – Verifica que està desactivat.
- `assert(hasClickAction())` – Verifica que el node és clicable.
- `assertTextEquals("text")` – Verifica que el text del node és exactament igual.
- `assertTextContains("text", ignoreCase = true)` – Verifica que el node conté el text especificat.
- `assertContentDescriptionEquals("descripció")` – Verifica que la `contentDescription` és correcta.



Instrumental UI Testing: actions

- **Actions:** aquests **mètodes** simulen accions de l'usuari.
 - `performClick()` – Simula un clic sobre el node.
 - `performScrollTo()` – Fa scroll fins que el node sigui visible.
 - `performTextInput("text")` – Simula l'entrada de text (per a `TextField`, etc.).
 - `performTextClearance()` – Esborra el text del node.
 - `performImeAction()` – Simula l'acció IME (Done, Next, etc.).
 - `performKeyPress(KeyEvent(...))` – Simula la pulsació d'una tecla.
 - `performTouchInput { ... }` – Permet simular gestos complexos com `swipe`, `drag`, etc.



Instrumental UI Testing

- Comprovem els estats inicials dels Composables:

```
@Test
fun checkInitialComposableValues () {
    composeTestRule.onNodeWithTag("initialText_id").assertTextEquals("Hello !")
    composeTestRule.onNodeWithTag("outlinedTextField_id").assertTextContains("
                                                                    , substring = false)
    composeTestRule.onNodeWithTag("counterText_id").assertTextEquals("0")
    composeTestRule.onNodeWithTag("checkBoxMulti_id").assertIsOff()
}
```



Instrumental UI Testing

- Comprovem l'efecte de prémer el check box:

```
@Test
fun checkCheckBoxMulti() {
    // Preparem test fent click al button:

    composeTestRule.onNodeWithTag("incrementButton_id").performClick()
    composeTestRule.onNodeWithTag("incrementButton_id").performClick()
    composeTestRule.onNodeWithTag("incrementButton_id").performClick()

    composeTestRule.onNodeWithTag("counterText_id").assertTextEquals("3")

    composeTestRule.onNodeWithTag("checkBoxMulti_id").performClick()
    composeTestRule.onNodeWithTag("counterText_id").assertTextEquals("6")
    ...
}
```



Instrumental UI Testing

```
...  
  
composeTestRule.onNodeWithTag("incrementButton_id").performClick()  
  
composeTestRule.onNodeWithTag("counterText_id").assertTextEquals("8")  
  
  
composeTestRule.onNodeWithTag("checkBoxMulti_id").performClick()  
  
composeTestRule.onNodeWithTag("counterText_id").assertTextEquals("4")  
  
  
composeTestRule.onNodeWithTag("checkBoxMulti_id").performClick()  
composeTestRule.onNodeWithTag("checkBoxMulti_id").performClick()  
composeTestRule.onNodeWithTag("checkBoxMulti_id").performClick()  
composeTestRule.onNodeWithTag("checkBoxMulti_id").performClick()  
composeTestRule.onNodeWithTag("counterText_id").assertTextEquals("4")  
  
}
```



Instrumental UI Testing

Run Context Menu:

- Run 'ViewInstrumentedUITe...'
- Debug 'ViewInstrumentedUITe...'
- Profiler: Run 'ViewInstrumentedUITe...' as profileable (low overhead)
- Profiler: Run 'ViewInstrumentedUITe...' as debuggable (complete data)
- Modify Run Configuration...

Test Results Table:

Test Results	Duration	Medium_Phone_API...
Test Results	30 s	6/6
ViewInstrumentedUITest	30 s	6/6
checkOutlinedTextField	8 s	✓
checkIncrementButton	5 s	✓
checkInitialComposableValues	3 s	✓
useAppContext	3 s	✓
checkCheckBoxMulti	6 s	✓
checkResetButton	3 s	✓

Build Output:

```
> Task :app:createDebugAndroidTestApkListingFileRedirect UP-TO-DATE
Connected to process 28359 on device 'Medium_Phone_API_35 [emulator-555]

> Task :app:connectedDebugAndroidTest
Starting 6 tests on Medium_Phone_API_35(AVD) - 15

Medium_Phone_API_35(AVD) - 15 Tests 1/6 completed. (0 skipped) (0 failed)
Medium_Phone_API_35(AVD) - 15 Tests 3/6 completed. (0 skipped) (0 failed)
Medium_Phone_API_35(AVD) - 15 Tests 5/6 completed. (0 skipped) (0 failed)
Finished 6 tests on Medium_Phone_API_35(AVD) - 15

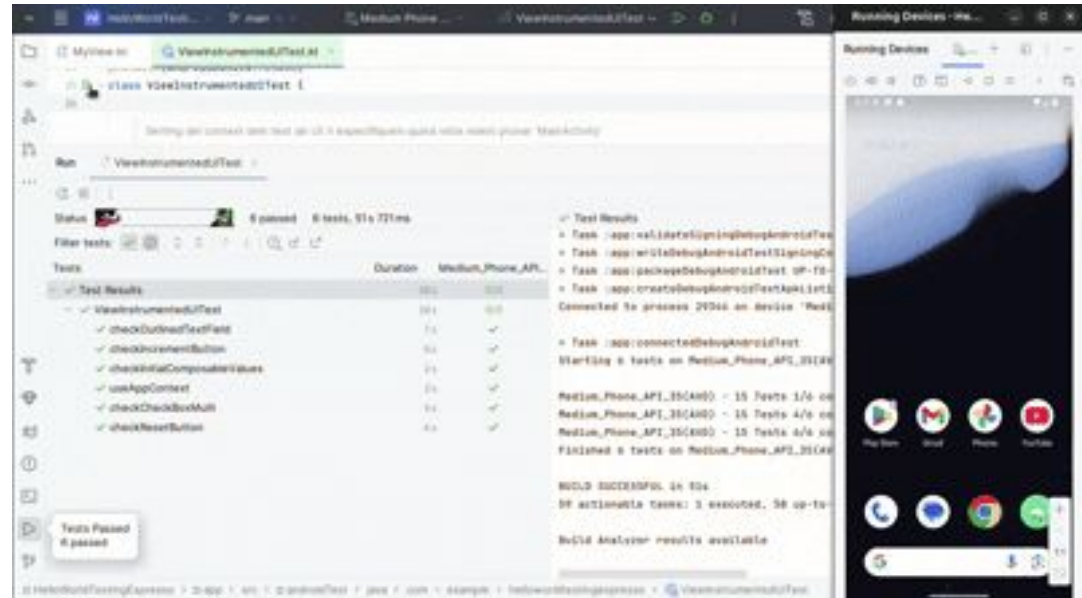
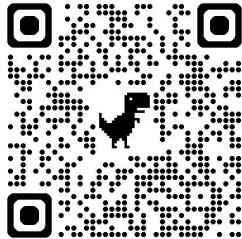
BUILD SUCCESSFUL in 57s
59 actionable tasks: 7 executed, 52 up-to-date

Build Analyzer results available
```



Instrumental UI Testing

- Link del repositori d'exemple:





Annex I: *build.gradle.kts*

```
dependencies {  
    // Plataforma de versions Compose  
    implementation(platform("androidx.compose:compose-bom:2024.04.01"))  
    implementation(platform(libs.androidx.compose.bom))  
    androidTestImplementation(platform(libs.androidx.compose.bom))  
  
    // _____  
    // Unit Testing (JVM)  
    // _____  
    testImplementation("androidx.arch.core:core-testing:2.2.0")  
    testImplementation(libs.junit)  
    ...  
}
```



Annex I: *build.gradle.kts*

```
// _____  
// Instrumental UI Testing (emulador/dispositiu)  
// _____  
  
androidTestImplementation("androidx.test.espresso:espresso-core"  
androidTestImplementation("androidx.test.ext:junit"  
androidTestImplementation("androidx.compose.ui:ui-test-junit4"  
androidTestImplementation(libs.androidx.espresso.core)  
androidTestImplementation(libs.androidx.junit)  
androidTestImplementation(libs.androidx.ui.test.junit4)
```

...



Annex I: *build.gradle.kts*

```
// _____  
// Debug Tools (només en debug)  
// _____  
  
debugImplementation("androidx.compose.ui:ui-tooling")  
debugImplementation("androidx.compose.ui:ui-test-manifest")  
  
debugImplementation(libs.androidx.ui.tooling)  
debugImplementation(libs.androidx.ui.test.manifest)  
  
...
```



Annex I: *build.gradle.kts*

```
// _____  
// Implementació principal de l'app  
// _____  
  
implementation("androidx.compose.runtime:runtime-livedata")  
implementation("androidx.compose.material3:material3")  
implementation("androidx.compose.ui:ui")  
implementation("androidx.lifecycle:lifecycle-runtime-ktx")  
implementation("androidx.activity:activity-compose")  
  
implementation(libs.androidx.core.ktx)  
implementation(libs.androidx.lifecycle.runtime.ktx)  
implementation(libs.androidx.activity.compose)  
implementation(libs.androidx.ui)  
implementation(libs.androidx.ui.graphics)  
implementation(libs.androidx.ui.tooling.preview)  
implementation(libs.androidx.material3)
```

```
}
```